Quiz 2 · responsible AI use for coding

Friday, May 29, 2026

Today: Friday May 29

Before class

- **No Perusall assignment**

Plan for today

- **Quiz 2** (first 20 minutes)
- How AI can get coding wrong
- Three verification skills
- In-class activity: AI-assisted analysis, with verification

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* · *DC* · ggplot2 reference

Quiz 2

Quiz 2: 20 minutes

- Closed-book, closed-notes, closed-internet, closed-AI.
- Covers everything from week 1 and the first half of week 2: data frames, `dplyr` pipes, ggplot2 basics, aesthetics, geoms, facets, stats, position adjustments.
- Five short questions. Show your work.

When you finish, turn in your quiz up front and we'll get started with the rest of class.

AI Use

AI Use in this Course

The course AI policy is on the syllabus.

- Academic Integrity & AI Policy

AI tools are allowed for this course, with disclosure. But "allowed" doesn't mean "use uncritically."

> Tip
>
> When you code with AI's help, **you are still the one responsible for the code** and you must be able to explain/defend each line you (or AI) submits.

What counts as "AI"

When I say "AI" today I mean things like:

- ChatGPT, Claude, Gemini (general chat-based assistants)
- GitHub Copilot, Cursor, Codeium (in-editor code completion)
- The "explain this code" / "fix this error" features inside RStudio extensions

The patterns we'll discuss apply across all of them. How can we use these tools without making mistakes we can't catch?

Why AI use can be particularly risky

The output of a good AI model **looks correct** even when it's wrong.

- Variable names are sensible.
- Syntax compiles or runs.
- The code resembles what a confident expert would write.
- It "explains itself" fluently when asked.

You can't catch a confident-sounding wrong answer by reading it the way you'd skim a textbook. You have to **actively verify**.

STAT 184 Skills

Here's why we are still teaching coding in a world with AI:

1. **You have to be able to read code**, which is even more important when you didn't write it yourself.
2. **You have to know what right looks like.** AI will gladly generate plausible (but-wrong) code; you need the expertise to spot it.

The skills you're building this summer (knowing what `group_by` does, how `aes()` works, what `mean(x, na.rm = TRUE)` means) are exactly the skills that let you catch AI mistakes.

How AI goes wrong

Common AI coding failures

This isn't an exhaustive list, but here's four general categories:

1. **Hallucinated functions and packages**: code that mentions things that don't exist
2. **Plausible-but-wrong logic**: code that runs and answers the wrong question
3. **Outdated knowledge**: code that worked years ago but doesn't now
4. **Misread context**: AI assumes things about your data that aren't true

Failure 1: Hallucinated functions

You ask: *"How do I make a violin plot in ggplot2 with the median marked?"*

AI confidently produces:

```
ggplot(penguins, aes(x = species, y = body_mass_g)) +
  geom_violin() +
  stat_median_marker(color = "red")  # <- this function does not exist
```

`stat_median_marker()` is not a real ggplot function. It sounds like it could be. The AI generated something *plausible*.

> ⚠ Warning
>
> **The actual answer** is `stat_summary(fun = median, geom = "point", color = "red")`. The fake function gives a clean error when you try to run it— but a busy student might paste it, see the error, and assume *they* did something wrong.

Failure 2: Plausible-but-wrong logic

You ask: *"What's the average flipper length for each species, in mm?"*

AI produces:

```
penguins |>
  group_by(species) |>
  summarize(avg_flipper = mean(flipper_length_mm))
```

```
# A tibble: 3 x 2
  species   avg_flipper
  <fct>           <dbl>
1 Adelie             NA
2 Chinstrap         196.
3 Gentoo             NA
```

That runs fine and gives you some numbers. **But two values are `NA`.**

Why? `flipper_length_mm` has missing values, and `mean()` returns `NA` by default if any input is `NA`. The right answer needs `na.rm = TRUE`. Since you didn't explicitly ask for this setting, AI made a choice itself and did not implement this step (even though it definitely knows about `na.rm`).

> **Warning**
>
> The potential issue here is that the output table *looks like an answer*. If you copy the numbers into a report without running the code yourself, you'll write "the average flipper length for Adélies is NA" and not notice.

Failure 3: Outdated knowledge

The tidyverse, the palmerpenguins', and even R itself changes. However, AI training data has a knowledge cutoff.

If you ask: *"How do I select multiple columns with a name pattern?"*

A model might give you:

```
penguins |>
  select(one_of("species", "island"))  # outdated; still works but discouraged
```

`one_of()` has been superseded by `any_of()` and `all_of()`. It still works (for now), but it's not the modern recommended approach.

A worse situation is a model confidently using functions that were *removed* in a recent version. This will likely throw errors with no obvious cause.

Failure 4: Misread context

You upload a CSV and ask: *"Plot the relationship between price and size."*

AI produces:

```
ggplot(data, aes(x = size, y = price)) +
  geom_point()
```

The AI cannot see your data. It is *guessing* about:

- Whether `data` is even the right object name
- Which column is "price" and which is "size" (what if they're named `$price` or `Price_USD` or `cost_dollars`?)
- Whether either is actually numeric
- Whether `size` is a continuous variable or a category like S/M/L

It assumed all of these in a way that *might* match your data and might not.

Verification skills

Three important verification skills

Whenever you use AI-generated code, you should be able to:

1. **Read it line by line and say what each line does**. For example, not just "filters the data" but *which* rows, on *which* condition.
2. **Verify the result against the data, not just the code**. Does the number make sense given what you know about the variable?
3. **Have a second (trustworthy) opinion ready**, such as the documentation (`?function_name`), the original textbook, the actual data itself, etc.

Skill 1: Read it before you trust it

If the code has a function you've never seen, **stop and look it up** using a trustworthy resource.

- `?function_name` in R
- The package's website (e.g., `dplyr.tidyverse.org`)

If you can't articulate what every line does to a classmate, **you are not yet ready to submit it**.

> **Tip**
>
> One thing you can try is taking a chunk of AI code, deleting all the comments, and writing your *own* comments above each line. If you get stuck on a line, that's the line to investigate.

Skill 2: Verify against the data

Before trusting summary numbers, **sanity-check against what you already know**.

The AI tells you the average penguin weighs 4200g.

- Is that reasonable? (A penguin? Yes, that's about right.)
- Did all rows contribute? (`nrow(penguins)` vs the rows used in the calculation)
- Are there `NA`s being silently dropped or silently propagated?

If the AI gives you a plot with three lines and your data has four species, **something is off**.

> **Warning**
>
> One of the more common "AI gave me wrong numbers" failure modes is **silent NA handling**. Either `na.rm = TRUE` was used when it shouldn't have been (treating missing data as if it didn't exist) or it wasn't used when it should have been (giving you NA when you wanted a number).

Skill 3: Have a second opinion ready

When the AI tells you something, ask: **does another source agree?**

- The R help page (`?function`)
- The R4DS / DataComputing textbooks
- A specific data source (the package documentation)

- The actual data, computed a different way

If you cannot verify a claim with *some* independent source, you don't actually know if it's true.

> **Note**
>
> This isn't unique to AI. It's just how you should treat any source you don't fully trust.

Disclosure

What disclosure looks like in this course

Every homework template has an "AI Disclosure" section near the top:

```
- [ ] No AI used
- [ ] Syntax / Debugging (e.g., finding a missing comma, fixing an error)
- [ ] Conceptual explanation (e.g., asking how ggplot() layers work)
- [ ] Code Generation (e.g., asking AI to write a function from scratch)
```

You should:

- Check every box that applies (it can be more than one)
- Briefly describe how you used it (1–2 sentences)

> **Tip**
>
> A good disclosure: *"Used Claude to debug a missing parenthesis in my mutate() call. Used Claude to explain what `after_stat(prop)` does."*
> A bad disclosure: leaving the box unchecked when you did use AI.

What counts as "using AI"

- **Asking for a code snippet**: yes, disclose
- **Asking for an explanation of an error**: yes, disclose
- **Asking conceptual questions like "what's the difference between filter and slice"**: yes, disclose
- **Spell-check / autocomplete in your editor**: no
- **Searching Google and reading Stack Overflow**: no
- **Reading the docs (`?function`)**: no

Activity

Activity: AI-assisted analysis, with verification

Roughly 25 minutes

- Activity Canvas Link

Format: I'll give you a moderately complex analysis question on a dataset. You'll:

1. **Try to answer it with AI assistance.** Use whatever AI tool you want.
2. **Check the result.** Find at least two things wrong, suspicious, or unverified about the AI's output.
3. **Fix what's broken.** Produce a verified, corrected version with documentation of what you found.

You **must** disclose what AI you used and how. The assignment is graded on the quality of your verification, not on whether your first attempt was correct.

What verification looks like

When you verify your own AI output, you're looking for:

- **Hallucinated functions**: does every function you used actually exist? Did you check `?function_name`?
- **Silent NA handling**: were rows dropped? Were NAs propagated?
- **Wrong grouping**: does the code group by what you actually wanted to group by?
- **Wrong filter**: does the code keep what you wanted to keep?
- **Unverified claims**: does the AI's *interpretation* of the result match what the numbers actually show?
- **Plausible nonsense**: does the result match what you'd expect given the data?

Write down what you found, what you changed, and why.

Wrap-up & looking ahead

Next Monday: Week 3 starts. We move from making plots to **wrangling data** (joins, pivots, the full `dplyr` verb set).

> **Tip**
>
> **Take-home for the weekend:** the next time you use AI for coding, try articulating to yourself what *every line* does before you run it. If you can't, that's the line to investigate further.

Upcoming Deadlines

- **Mon 6/1, before class**: *DC* §7.1–7.7
- **Fri 5/29, 11:59 p.m. (Tonight)**: HW 2: Pipes, Plots & Reading Graphics

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* · *DC* · ggplot2 reference